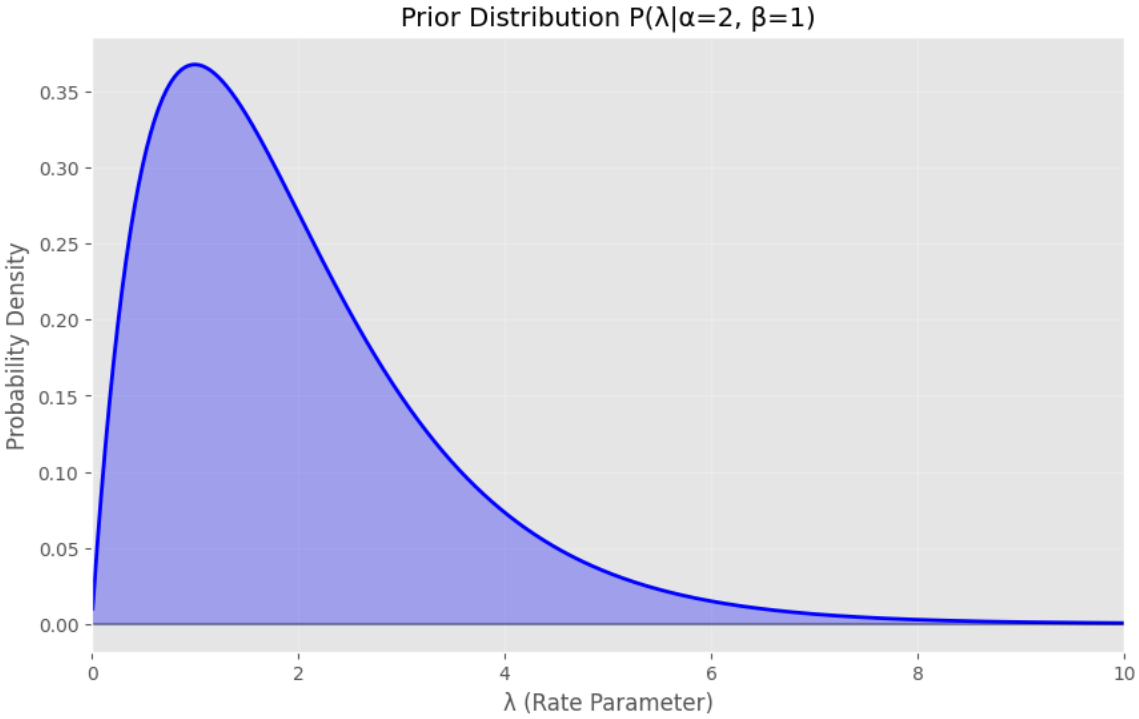
Student name and number : Alex Carter, 240355490

By submitting my assignment: I declare that I have personally prepared this assignment. The work is my own, carried out personally by me unless otherwise stated and has not been generated using Artificial Intelligence tools unless specified as a clearly stated approved component of the assessment brief. All sources of information, including quotations, are acknowledged by means of the appropriate citations and references. I declare that this work has not gained credit previously for another module at this or another University. I understand that plagiarism, collusion, copying another student and commissioning (which for the avoidance of doubt includes the use of essay mills and other paid for assessment writing services, as well as unattributed use of work generated by Artificial Intelligence tools) are regarded as offences against the University's Assessment Regulations and may result in formal disciplinary proceedings. I understand that by submitting this assessment, I declare myself fit to be able to undertake the assessment and accept the outcome of the assessment as valid.

Question 1:

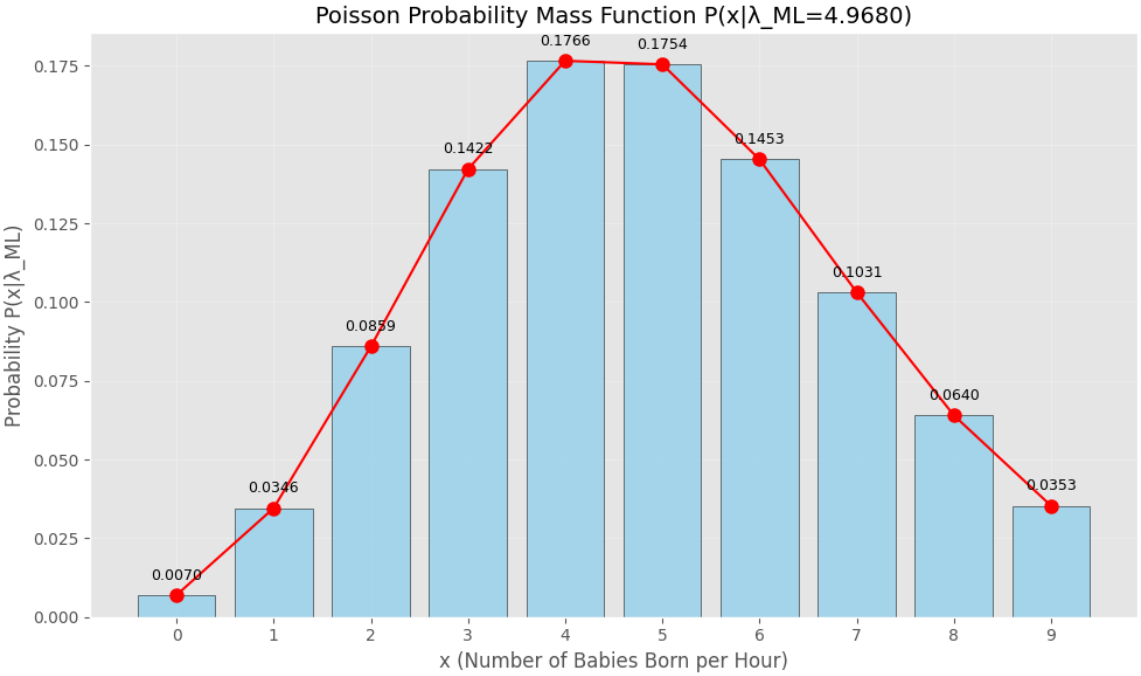
a) The data follows a Poisson distribution.



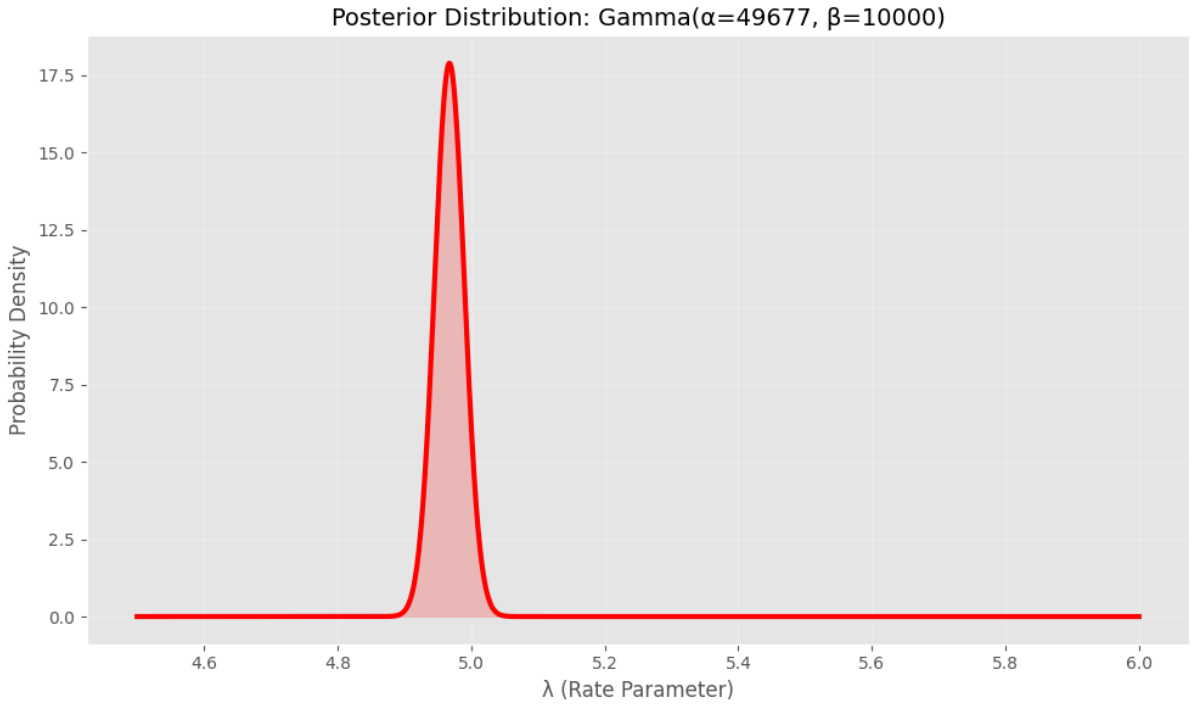
b)

c) λ_ML = 4.9680

d)

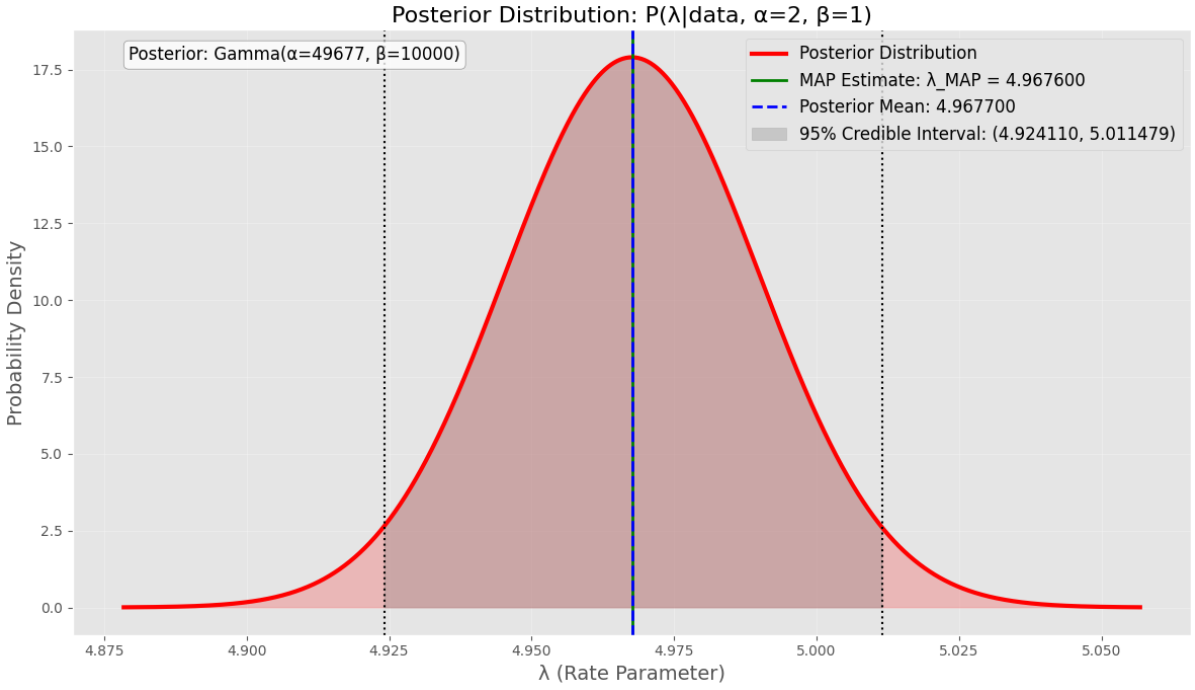


e) Posterior Statistics: Mean: 4.967700 Mode: 4.967600 95% Credible Interval: (4.924110, 5.011479)

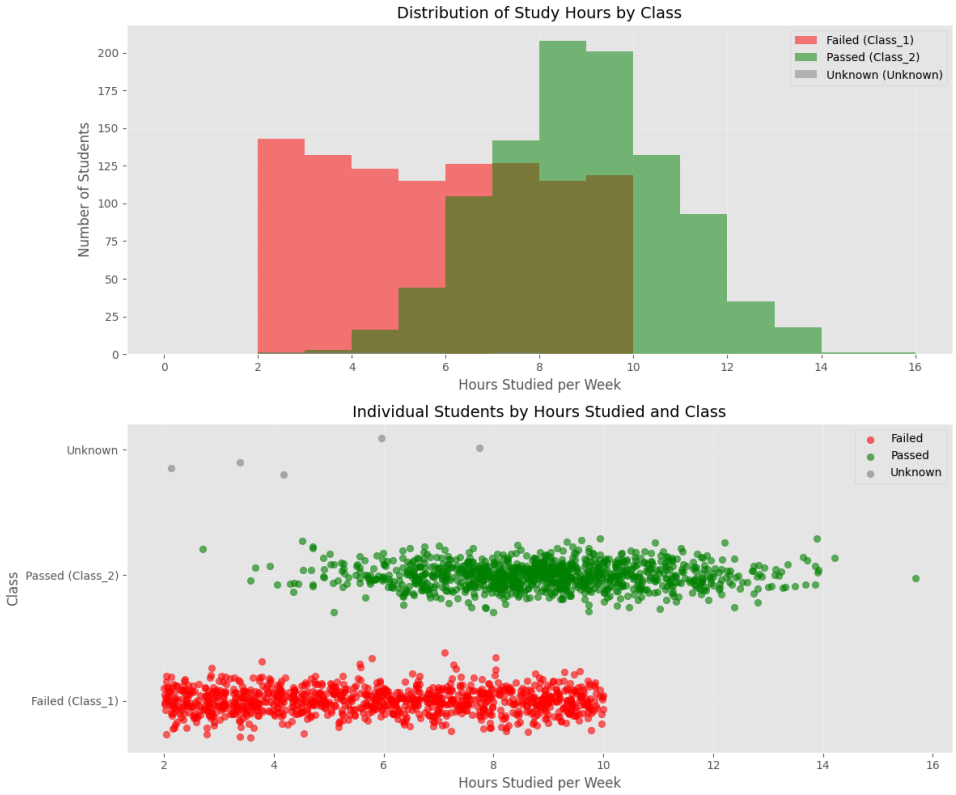


f) MAP estimate (λ_MAP) = 4.967600

g)



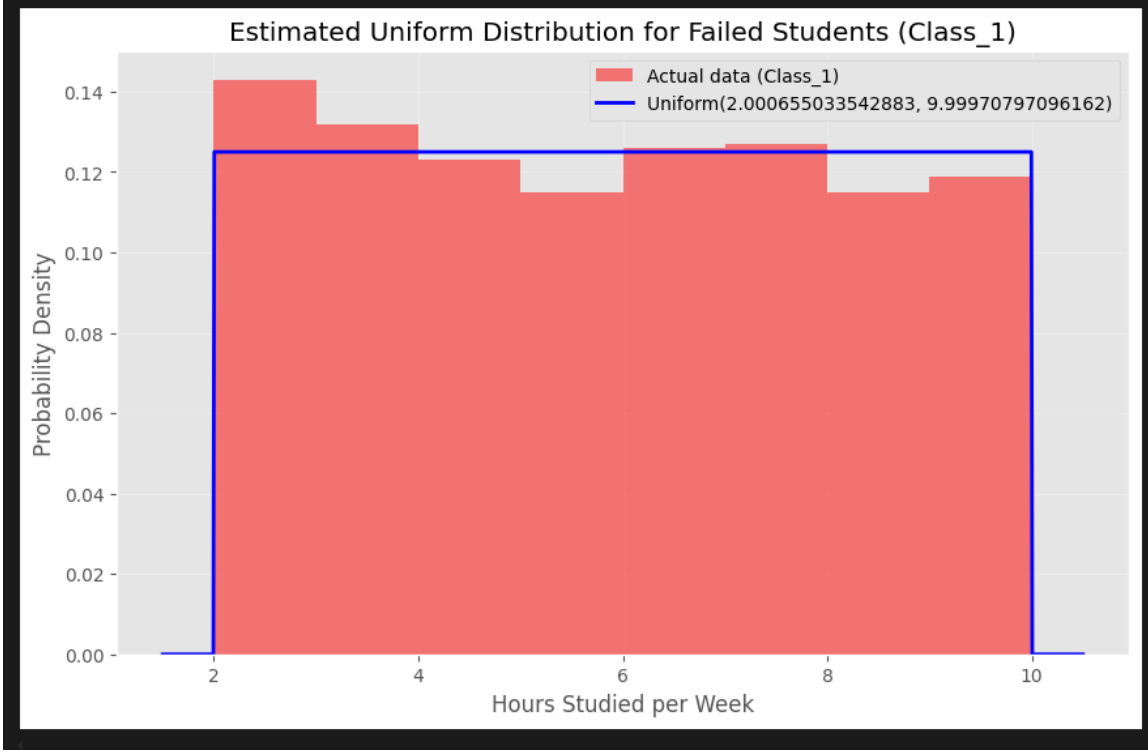
Question 2:



a)

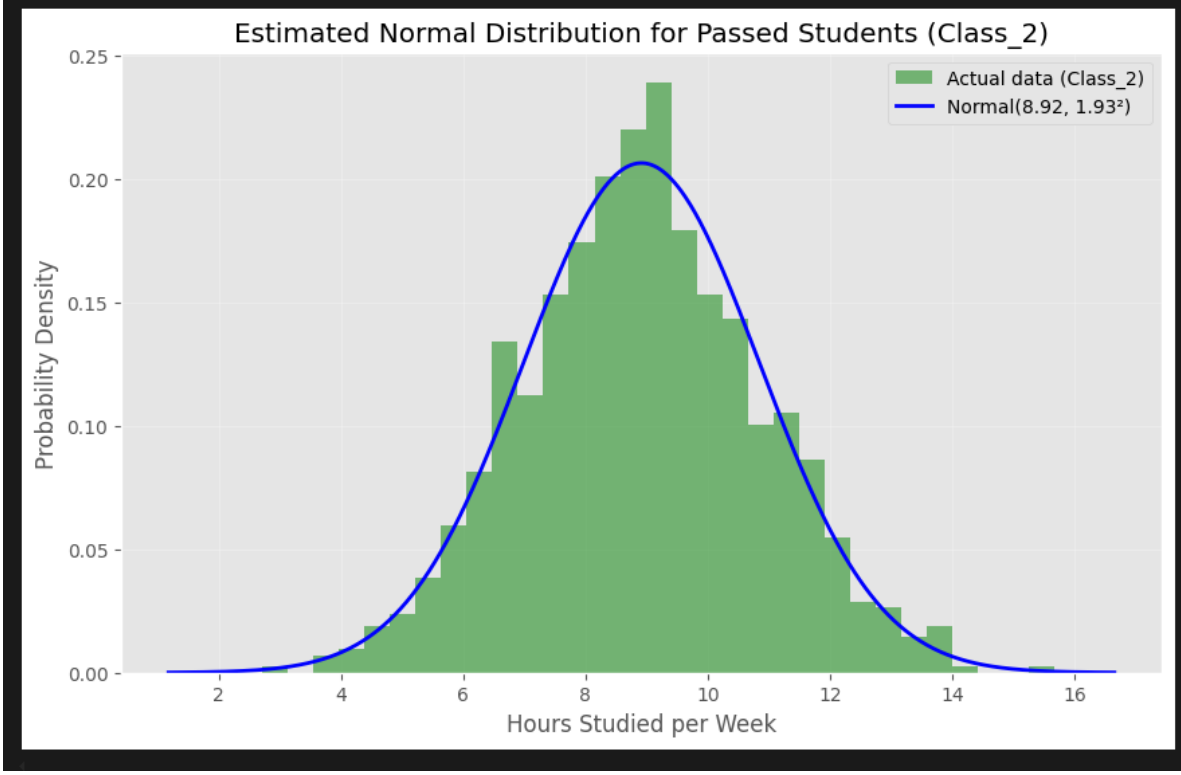
b) Uniform distribution parameters for failed students: a (minimum) = 2.000655033542883 b (maximum) = 9.99970797096162

```
Uniform distribution parameters for failed students:
a (minimum) = 2.000655033542883
b (maximum) = 9.99970797096162
```

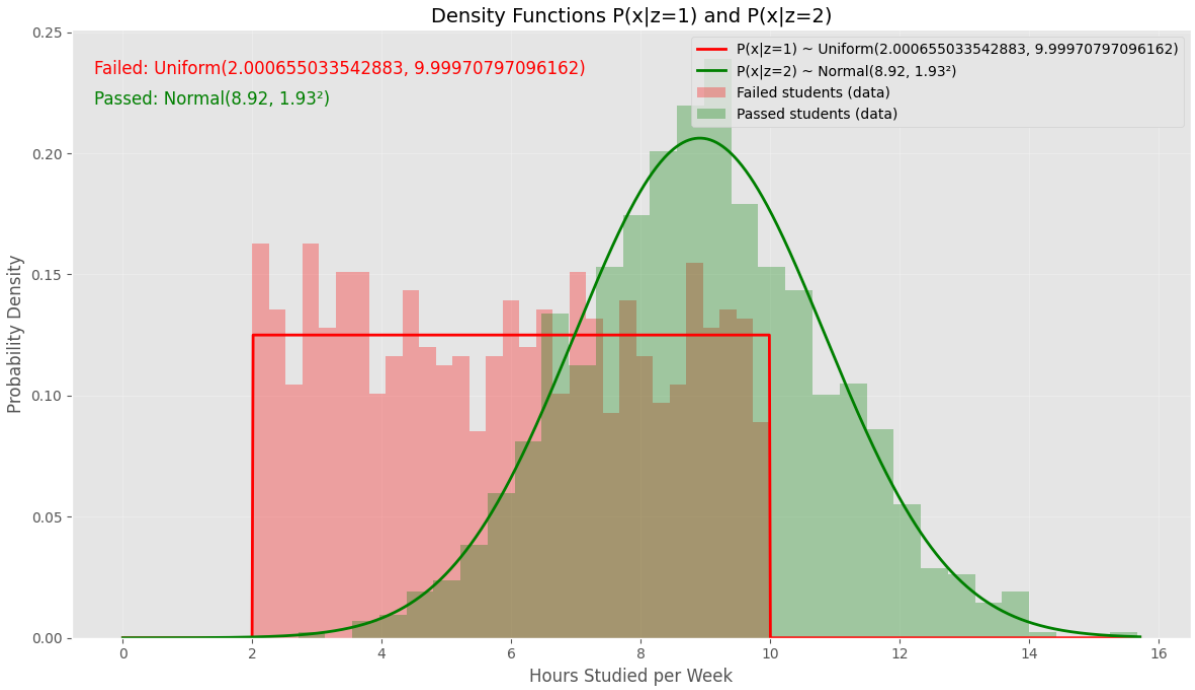


c)

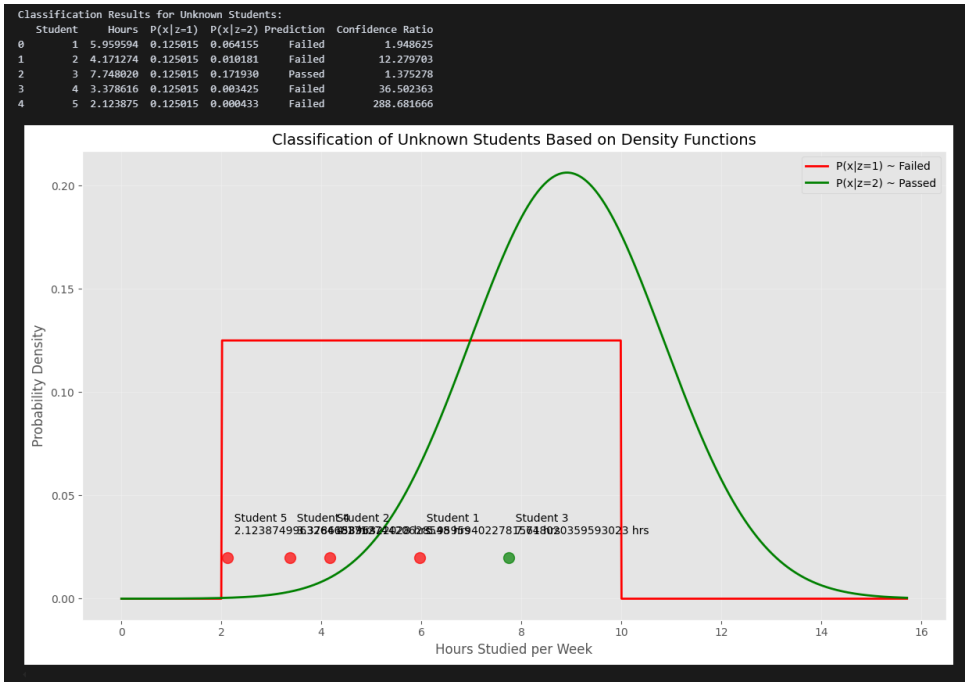
```
Normal distribution parameters for passed students:
μ (mean) = 8.9154
σ (standard deviation) = 1.9339
```



d)



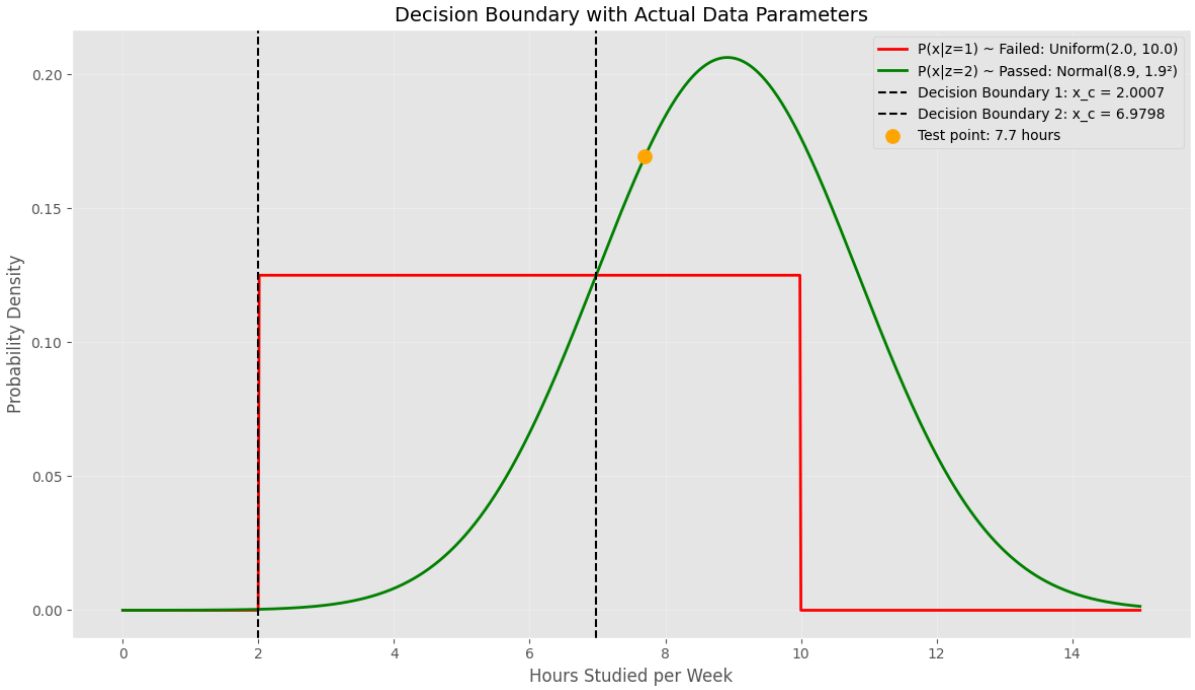
e)



Classification Results for Unknown Students:

	Student	Hours	P(x|z=1)	P(x|z=2)	Prediction	Confidence Ratio
0	1	5.959594	0.125015	0.064155	Failed	1.948625
1	2	4.171274	0.125015	0.010181	Failed	12.279703
2	3	7.748020	0.125015	0.171930	Passed	1.375278
3	4	3.378616	0.125015	0.003425	Failed	36.502363
4	5	2.123875	0.125015	0.000433	Failed	288.681666

f)

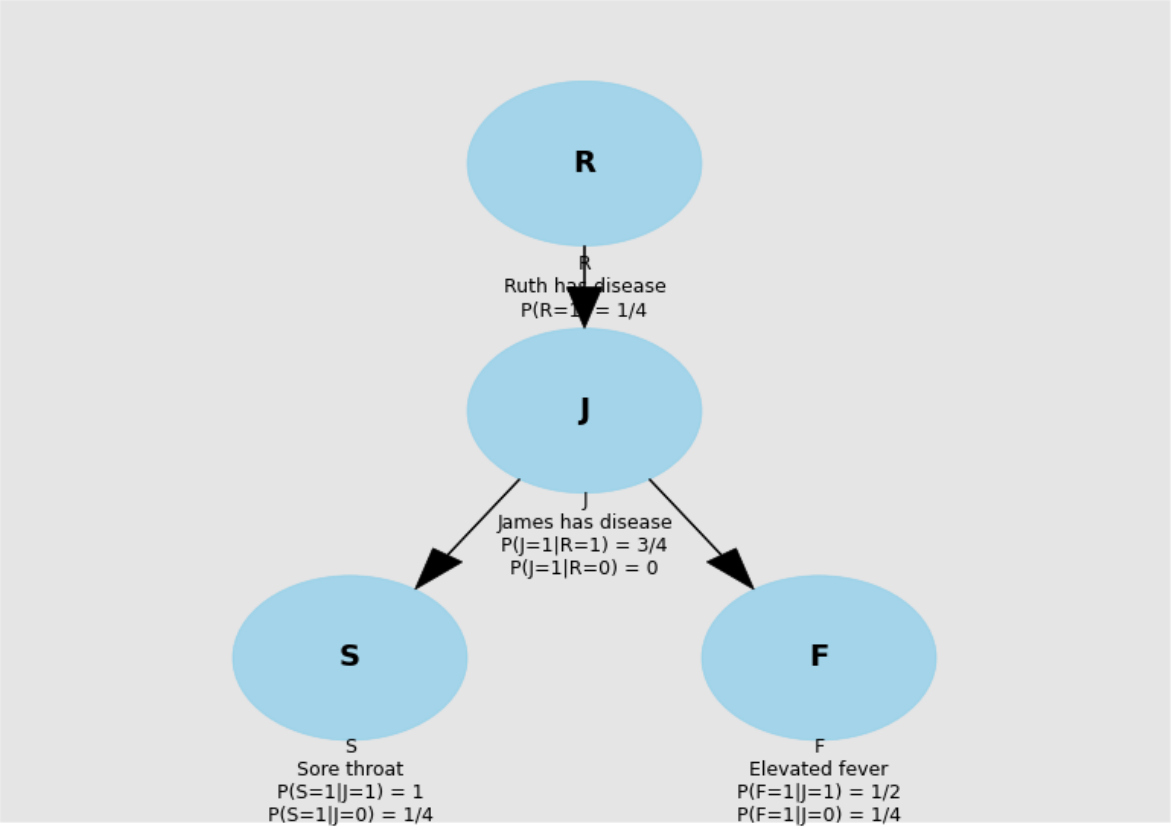


g) `Misclassification error: 0.3776 or 37.76%`

Question 3:

a)

Directed Acyclic Graph: Disease Transmission Model



b)

Joint Probability Factorization:

$P(R, J, S, F) = P(R) \times P(J|R) \times P(S|J) \times P(F|J)$

Probabilities Given in the Question:

Prior Probability:

$P(R = 1) = \frac{1}{4}$

$P(R = 0) = \frac{3}{4}$

Probability of James Being Infected:

$P(J = 1|R = 1) = \frac{3}{4}$

$P(J = 0|R = 1) = \frac{1}{4}$

$P(J = 1|R = 0) = 0$

$P(J = 0|R = 0) = 1$

Probability of Sore Throat:

$P(S = 1|J = 1) = 1$

$P(S = 0|J = 1) = 0$

$P(S = 1|J = 0) = \frac{1}{4}$

$P(S = 0|J = 0) = \frac{3}{4}$

Probability of Elevated Fever:

$P(F = 1|J = 1) = \frac{1}{2}$

$P(F = 0|J = 1) = \frac{1}{2}$

$P(F = 1|J = 0) = \frac{1}{4}$

$P(F = 0|J = 0) = \frac{3}{4}$

Example Calculation:

$P(R = 1, J = 1, S = 1, F = 1) = P(R = 1) \times P(J = 1|R = 1) \times P(S = 1|J = 1) \times P(F = 1|J = 1)$

$P(R = 1, J = 1, S = 1, F = 1) = \frac{1}{4} \times \frac{3}{4} \times 1 \times \frac{1}{2} = \frac{3}{32}$

c)

Step 1 : Calculate $P(S = 1, F = 1|R = 1)$

$P(S = 1, F = 1|R = 1) = P(S = 1|J = 1) \times P(F = 1|J = 1) \times P(J = 1|R = 1) + P(S = 1|J = 0) \times P(F = 1|J = 0) \times P(J = 0|R = 1)$

$P(S = 1, F = 1|R = 1) = 1 \times \frac{1}{2} \times \frac{3}{4} + \frac{1}{4} \times \frac{1}{4} \times \frac{1}{4}$

$P(S = 1, F = 1|R = 1) = \frac{3}{8} + \frac{1}{64} = \frac{24}{64} + \frac{1}{64} = \frac{25}{64}$

Step 2: Calculate $P(S = 1, F = 1|R = 0)$

$P(S = 1, F = 1|R = 0) = 1 \times \frac{1}{2} \times 0 + \frac{1}{4} \times \frac{1}{4} \times 1 = \frac{1}{16}$

Step 3 : Calculate $P(S = 1, F = 1)$

$P(S = 1, F = 1) = P(S = 1, F = 1|R = 1) \times P(R = 1) + P(S = 1, F = 1|R = 0) \times P(R = 0)$

$P(S = 1, F = 1) = \frac{25}{64} \times \frac{1}{4} + \frac{1}{16} \times \frac{3}{4}$

$P(S = 1, F = 1) = \frac{25}{256} + \frac{3}{64} = \frac{25}{256} + \frac{12}{256} = \frac{37}{256}$

Step 4 : Apply Bayestheorem

$P(R = 1|S = 1, F = 1) = \frac{P(S = 1, F = 1|R = 1) \times P(R = 1)}{P(S = 1, F = 1)}$

$P(R = 1|S = 1, F = 1) = \frac{\frac{25}{64} \times \frac{1}{4}}{\frac{37}{256}}$

$P(R = 1|S = 1, F = 1) = \frac{\frac{25}{256}}{\frac{37}{256}} = \frac{25}{37}$

$\boxed{P(R = 1|S = 1, F = 1) = \frac{25}{37}}$

d)

Find $P(R=1|S=0,F=1)$

$$P(R=1|S=0,F=1)=\frac{P(S=0,F=1|R=1)\times P(R=1)}{P(S=0,F=1)}$$

Step 1: Calculate $P(S=0,F=1|R=1)$

$$P(S=0,F=1|R=1)=\sum_{j\in\{0,1\}}P(S=0|J=j)\times P(F=1|J=j)\times P(J=j|R=1)$$

$$P(S=0,F=1|R=1)=P(S=0|J=1)\times P(F=1|J=1)\times P(J=1|R=1)+P(S=0|J=0)\times P(F=1|J=0)\times P(J=0|R=1)$$

$$P(S=0,F=1|R=1)=0\times\frac{1}{2}\times\frac{3}{4}+\frac{3}{4}\times\frac{1}{4}\times\frac{1}{4}$$

$$P(S=0,F=1|R=1)=0+\frac{3\times1\times1}{4\times4\times4}=\frac{3}{64}$$

Step 2: Calculate $P(S=0,F=1|R=0)$

$$P(S=0,F=1|R=0)=P(S=0|J=1)\times P(F=1|J=1)\times P(J=1|R=0)+P(S=0|J=0)\times P(F=1|J=0)\times P(J=0|R=0)$$

$$P(S=0,F=1|R=0)=0\times\frac{1}{2}\times0+\frac{3}{4}\times\frac{1}{4}\times1$$

$$P(S=0,F=1|R=0)=0+\frac{3}{16}=\frac{3}{16}$$

Step 3: Calculate $P(S=0,F=1)$

$$P(S=0,F=1)=P(S=0,F=1|R=1)\times P(R=1)+P(S=0,F=1|R=0)\times P(R=0)$$

$$P(S=0,F=1)=\frac{3}{64}\times\frac{1}{4}+\frac{3}{16}\times\frac{3}{4}$$

$$P(S=0,F=1)=\frac{3}{256}+\frac{9}{64}=\frac{3}{256}+\frac{36}{256}=\frac{39}{256}$$

Step 4: Apply Bayesťheorem

$$P(R=1|S=0,F=1)=\frac{P(S=0,F=1|R=1)\times P(R=1)}{P(S=0,F=1)}$$

$$P(R=1|S=0,F=1)=\frac{\frac{3}{64}\times\frac{1}{4}}{\frac{39}{256}}$$

$$P(R=1|S=0,F=1)=\frac{\frac{3}{256}}{\frac{39}{256}}=\frac{3}{39}=\frac{1}{13}$$

$$\boxed{P(R=1|S=0,F=1)=\frac{1}{13}}$$

e)

Find $P(J=1|F=1,T=1)$ (probability James is ill given fever and Ruthś positive test)

$$P(J=1|F=1,T=1)=\frac{P(F=1|J=1,T=1)\times P(J=1|T=1)}{P(F=1|T=1)}$$

Step 1: Calculate $P(R=1|T=1)$ (probability Ruth has disease given positive test)

$$P(R=1|T=1)=\frac{P(T=1|R=1)\times P(R=1)}{P(T=1)}$$

$$P(T=1)=P(T=1|R=1)\times P(R=1)+P(T=1|R=0)\times P(R=0)$$

$$P(T=1)=\frac{3}{4}\times\frac{1}{4}+\frac{1}{4}\times\frac{3}{4}=\frac{3}{16}+\frac{3}{16}=\frac{6}{16}=\frac{3}{8}$$

$$P(R=1|T=1)=\frac{\frac{3}{4}\times\frac{1}{4}}{\frac{3}{8}}=\frac{\frac{3}{16}}{\frac{3}{8}}=\frac{1}{2}$$

Step 2: Calculate $P(J=1|T=1)$ (probability James is ill given Ruthś positive test)

$$P(J=1|T=1)=P(J=1|R=1)\times P(R=1|T=1)+P(J=1|R=0)\times P(R=0|T=1)$$

$$P(J=1|T=1)=\frac{3}{4}\times\frac{1}{2}+0\times\frac{1}{2}=\frac{3}{8}$$

Step 3: Calculate $P(F=1|T=1)$ (probability James has fever given Ruthś positive test)

$$P(F=1|T=1)=P(F=1|J=1)\times P(J=1|T=1)+P(F=1|J=0)\times P(J=0|T=1)$$

$$P(F=1|T=1)=\frac{1}{2}\times\frac{3}{8}+\frac{1}{4}\times\frac{5}{8}=\frac{3}{16}+\frac{5}{32}=\frac{6}{32}+\frac{5}{32}=\frac{11}{32}$$

Step 4: Apply Bayesťheorem

$$P(J=1|F=1,T=1)=\frac{P(F=1|J=1)\times P(J=1|T=1)}{P(F=1|T=1)}$$

$$P(J=1|F=1,T=1)=\frac{\frac{1}{2}\times\frac{3}{8}}{\frac{11}{32}}=\frac{\frac{3}{16}}{\frac{11}{32}}=\frac{3}{16}\times\frac{32}{11}=\frac{96}{176}=\frac{6}{11}$$

$$\boxed{P(J=1|F=1,T=1)=\frac{6}{11}}$$

Appendix:

Code for section 1, a):

```python
#Question 1, a

# Set the style
plt.style.use('ggplot')

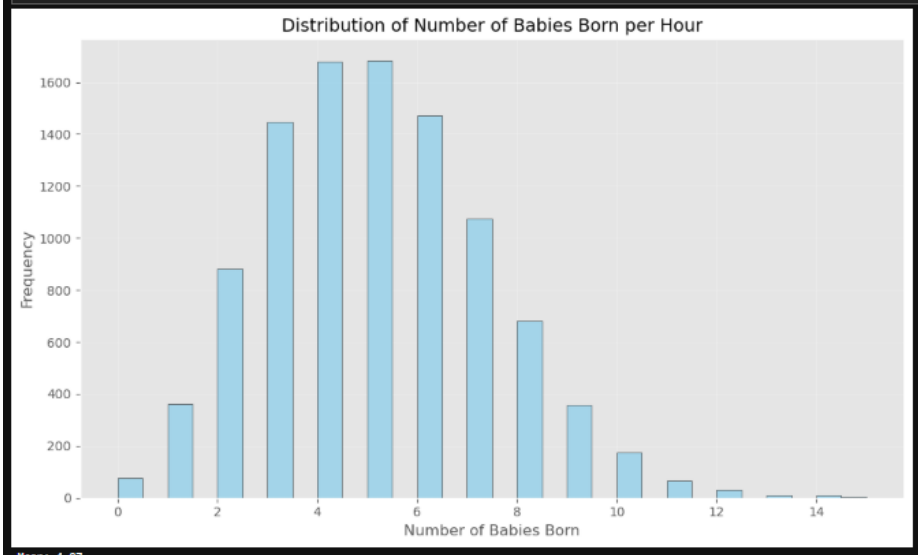
# Load the hospital data
hospital_data = pd.read_csv('Hospital.csv')

# Create a histogram of the data
plt.figure(figsize=(10, 6))
plt.hist(hospital_data.iloc[:, 0], bins=30, alpha=0.7, density=False,
         edgecolor='black', color='skyblue')


plt.title('Distribution of Number of Babies Born per Hour', fontsize=14)
plt.xlabel('Number of Babies Born', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('hospital_histogram.png', dpi=300)
plt.show()

# Calculate basic statistics for analysis
mean_value = hospital_data.iloc[:, 0].mean()
variance = hospital_data.iloc[:, 0].var()
mode_value = hospital_data.iloc[:, 0].mode()[0]
median_value = hospital_data.iloc[:, 0].median()

print(f"Mean: {mean_value:.2f}")
print(f"Variance: {variance:.2f}")
print(f"Mode: {mode_value}")
print(f"Median: {median_value}")
```




```
Mean: 4.97
Variance: 5.02
Mode: 5
Median: 5.0
```

Code for question 1, b):

```python
# Define the parameters
alpha = 2
beta = 1

# Generate λ values in the range (0, 10]
lambda_values = np.linspace(0.01, 10, 1000)  # Starting from 0.01 to avoid division by zero

# Calculate the gamma distribution PDF
gamma_pdf = stats.gamma.pdf(lambda_values, a=alpha, scale=1/beta)

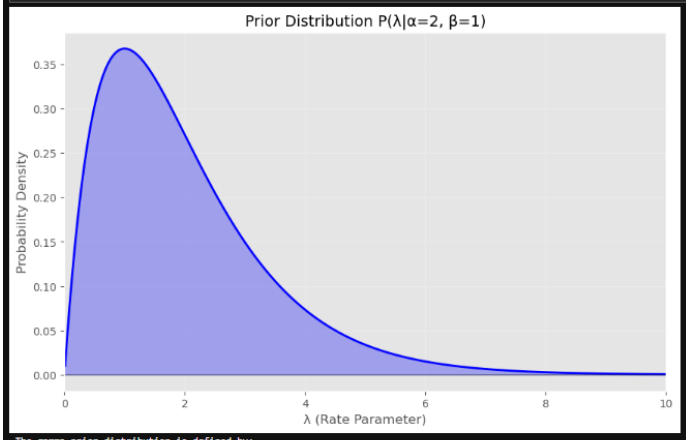
# Plot the gamma distribution
plt.figure(figsize=(10, 6))
plt.plot(lambda_values, gamma_pdf, 'b-', linewidth=2)
plt.fill_between(lambda_values, gamma_pdf, alpha=0.3, color='blue')
plt.title(f'Prior Distribution P(λ|α={alpha}, β={beta})', fontsize=14)
plt.xlabel('λ (Rate Parameter)', fontsize=12)
plt.ylabel('Probability Density', fontsize=12)
plt.grid(True, alpha=0.3)
plt.xlim(0, 10)
plt.axhline(y=0, color='k', linestyle='-', alpha=0.3)
plt.savefig('gamma_prior.png', dpi=300)
plt.show()

# Display the analytical form of the PDF
from IPython.display import display, Math

# Format the Gamma PDF equation
equation = r"P(\lambda|\alpha,\beta) = \frac{\beta^\alpha}{\Gamma(\alpha)}\lambda^{\alpha-1}e^{-\beta\lambda}"
print("The gamma prior distribution is defined by:")
display(Math(equation))
print(f"With α = {alpha} and β = {beta}")

# Calculate some important statistics of this distribution
mean = alpha / beta
mode = (alpha - 1) / beta if alpha > 1 else "Undefined"
variance = alpha / (beta**2)

print(f"\nStatistics of this prior distribution:")
print(f"Mean: {mean}")
print(f"Mode: {mode}")
print(f"Variance: {variance}")
```




```
The gamma prior distribution is defined by:
```

$$P(\lambda|\alpha,\beta) = \frac{\beta^\alpha}{\Gamma(\alpha)}\lambda^{\alpha-1}e^{-\beta\lambda}$$

```
With α = 2 and β = 1

Statistics of this prior distribution:
Mean: 2.0
Mode: 1.0
Variance: 2.0
```

Question 1 c&d) :

```python
#Question 1 c&d)

# Load the hospital data
hospital_data = pd.read_csv('Hospital.csv')

# Calculate the maximum likelihood estimate for lambda (for part c)
lambda_ML = hospital_data.iloc[:, 0].mean()
print(f"Maximum Likelihood Estimate (λ_ML): {lambda_ML:.4f}")

# Define the x values as specified
x_values = np.arange(0, 10)  # 0 to 9

# Calculate P(x|λ_ML) for each x
pmf_values = stats.poisson.pmf(x_values, lambda_ML)

# Create the plot
plt.figure(figsize=(10, 6))

# Bar plot for the Poisson PMF
plt.bar(x_values, pmf_values, color='skyblue', edgecolor='black', alpha=0.7)

# Scatter plot with connecting lines to emphasize discrete nature
plt.plot(x_values, pmf_values, 'ro-', ms=8)

# Customize the plot
plt.title(f'Poisson Probability Mass Function P(x|λ_ML={lambda_ML:.4f})', fontsize=14)
plt.xlabel('x (Number of Babies Born per Hour)', fontsize=12)
plt.ylabel('Probability P(x|λ_ML)', fontsize=12)
plt.xticks(x_values)
plt.grid(True, alpha=0.3)

# Add text labels for the probabilities
for i, prob in enumerate(pmf_values):
    plt.text(i, prob + 0.005, f"{prob:.4f}", ha='center', fontsize=9)

# Create a table of values
table_data = [x_values, np.round(pmf_values, 4)]
rows = ['x', 'P(x|λ_ML)']
print("\nProbability Mass Function Values:")
for i in range(len(rows)):
    print(f"{rows[i]}: {table_data[i]}")

plt.tight_layout()
plt.savefig('poisson_pmf.png', dpi=300)
plt.show()
```

```
Maximum Likelihood Estimate (λ_ML): 4.9680

Probability Mass Function Values:
x: [0 1 2 3 4 5 6 7 8 9]
P(x|λ_ML): [0.007  0.0346 0.0859 0.1422 0.1766 0.1754 0.1453 0.1031 0.064  0.0353]
```

Code for question 1, e):

```python
#Question 1, e)

# Load the hospital data
hospital_data = pd.read_csv('Hospital.csv')
data = hospital_data.iloc[:, 0]

# Prior parameters
alpha_prior = 2
beta_prior = 1

# Calculate posterior parameters
n = len(data)
sum_x = data.sum()
alpha_posterior = alpha_prior + sum_x
beta_posterior = beta_prior + n

# Display the computed posterior distribution
print("\nComputed Posterior Distribution:")
print("--------------------------------")
print(f"P(λ|data, α, β) = Gamma(λ; α'={alpha_posterior}, β'={beta_posterior})")
print(f"P(λ|data, α, β) = {beta_posterior}^{alpha_posterior} / Γ({alpha_posterior}) × λ^({alpha_posterior-1}) × e^(-{beta_posterior}λ)")

# Calculate key statistics
posterior_mean = alpha_posterior / beta_posterior
posterior_mode = (alpha_posterior - 1) / beta_posterior
posterior_variance = alpha_posterior / (beta_posterior ** 2)
posterior_std = np.sqrt(posterior_variance)

print("\nPosterior Statistics:")
print(f"Mean: {posterior_mean:.6f}")
print(f"Mode: {posterior_mode:.6f}")
print(f"95% Credible Interval: ({stats.gamma.ppf(0.025, alpha_posterior, scale=1/beta_posterior):.6f}, "
      f"{stats.gamma.ppf(0.975, alpha_posterior, scale=1/beta_posterior):.6f})")

# Create a simple, clear plot of the posterior distribution
lambda_values = np.linspace(4.5, 6.0, 1000)  # Focus on relevant range
posterior_pdf = stats.gamma.pdf(lambda_values, a=alpha_posterior, scale=1/beta_posterior)

plt.figure(figsize=(10, 6))
plt.plot(lambda_values, posterior_pdf, 'r-', linewidth=3)
plt.fill_between(lambda_values, posterior_pdf, alpha=0.2, color='red')
plt.title(f'Posterior Distribution: Gamma(α={alpha_posterior}, β={beta_posterior})', fontsize=14)
plt.xlabel('λ (Rate Parameter)', fontsize=12)
plt.ylabel('Probability Density', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('posterior_distribution_clean.png', dpi=300)
plt.show()
```

Code for question 1, f) :

```python
#Code for question 1, f)

# Load the hospital data
hospital_data = pd.read_csv('Hospital.csv')
data = hospital_data.iloc[:, 0]

# Prior parameters (given)
alpha_prior = 2
beta_prior = 1

# Calculate posterior parameters
n = len(data)
sum_x = data.sum()
alpha_posterior = alpha_prior + sum_x
beta_posterior = beta_prior + n

# Calculate MAP estimate (posterior mode)
lambda_MAP = (alpha_posterior - 1) / beta_posterior

# Calculate MLE for comparison
lambda_ML = data.mean()

print(f"Prior parameters: α = {alpha_prior}, β = {beta_prior}")
print(f"\nPosterior parameters:")
print(f"α' = α + Σx = {alpha_prior} + {sum_x} = {alpha_posterior}")
print(f"β' = β + n = {beta_prior} + {n} = {beta_posterior}")
print(f"\nMAP estimate (λ_MAP) = (α' - 1)/β' = ({alpha_posterior} - 1)/{beta_posterior} = {lambda_MAP:.6f}")
print(f"MLE estimate (λ_ML) = {lambda_ML:.6f}")
print(f"Difference (MAP - MLE) = {lambda_MAP - lambda_ML:.6f}")

# Visualize the posterior with the MAP estimate
lambda_values = np.linspace(5.15, 5.25, 1000)  # Focus on a narrow range around the MAP
posterior_pdf = stats.gamma.pdf(lambda_values, a=alpha_posterior, scale=1/beta_posterior)

plt.figure(figsize=(10, 6))
plt.plot(lambda_values, posterior_pdf, 'r-', linewidth=3, label='Posterior Distribution')
plt.axvline(x=lambda_MAP, color='g', linestyle='-', linewidth=2, label=f'MAP Estimate: λ_MAP = {lambda_MAP:.6f}')
plt.axvline(x=lambda_ML, color='b', linestyle='--', linewidth=2, label=f'ML Estimate: λ_ML = {lambda_ML:.6f}')

plt.title('Posterior Distribution with MAP and ML Estimates', fontsize=14)
plt.xlabel('λ (Rate Parameter)', fontsize=12)
plt.ylabel('Probability Density', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend()

plt.tight_layout()
plt.savefig('map_estimate.png', dpi=300)
plt.show()
```

```
Prior parameters: α = 2, β = 1
```

Code for question 1, g) :

```python
#Question 1, g)

# Load the hospital data
hospital_data = pd.read_csv('Hospital.csv')
data = hospital_data.iloc[:, 0]

# Prior parameters (given)
alpha_prior = 2
beta_prior = 1

# Calculate posterior parameters
n = len(data)
sum_x = data.sum()
alpha_posterior = alpha_prior + sum_x
beta_posterior = beta_prior + n

# Calculate the MAP and mean of the posterior for reference
lambda_MAP = (alpha_posterior - 1) / beta_posterior
lambda_mean = alpha_posterior / beta_posterior

print(f"Posterior distribution: Gamma(α'={alpha_posterior}, β'={beta_posterior})")
print(f"MAP estimate: λ_MAP = {lambda_MAP:.6f}")
print(f"Posterior mean: E[λ|data] = {lambda_mean:.6f}")

# Calculate a reasonable domain for plotting
# Since we have many data points, the posterior will be concentrated
# A good approach is to center around the MAP and extend a few standard deviations
posterior_std = np.sqrt(alpha_posterior / (beta_posterior**2))
min_lambda = max(0.001, lambda_MAP - 4*posterior_std)  # Avoid going below zero
max_lambda = lambda_MAP + 4*posterior_std

# Generate lambda values for plotting
lambda_values = np.linspace(min_lambda, max_lambda, 1000)

# Calculate the posterior PDF
posterior_pdf = stats.gamma.pdf(lambda_values, a=alpha_posterior, scale=1/beta_posterior)

# Create the plot
plt.figure(figsize=(12, 7))

# Plot the posterior distribution
plt.plot(lambda_values, posterior_pdf, 'r-', linewidth=3, label='Posterior Distribution')
plt.fill_between(lambda_values, posterior_pdf, alpha=0.2, color='red')

# Add vertical lines for key values
plt.axvline(x=lambda_MAP, color='g', linestyle='-', linewidth=2,
            label=f'MAP Estimate: λ_MAP = {lambda_MAP:.6f}')
plt.axvline(x=lambda_mean, color='b', linestyle='--', linewidth=2,
            label=f'Posterior Mean: {lambda_mean:.6f}')

# Add a 95% credible interval
lower_ci = stats.gamma.ppf(0.025, alpha_posterior, scale=1/beta_posterior)
upper_ci = stats.gamma.ppf(0.975, alpha_posterior, scale=1/beta_posterior)
plt.axvline(x=lower_ci, color='k', linestyle=':', linewidth=1.5)
plt.axvline(x=upper_ci, color='k', linestyle=':', linewidth=1.5)
plt.fill_between(lambda_values, posterior_pdf,
                 where=((lambda_values >= lower_ci) & (lambda_values <= upper_ci)),
                 alpha=0.3, color='gray',
                 label=f'95% Credible Interval: ({lower_ci:.6f}, {upper_ci:.6f})')

# Add labels and title
plt.title('Posterior Distribution: P(λ|data, α=2, β=1)', fontsize=16)
plt.xlabel('λ (Rate Parameter)', fontsize=14)
plt.ylabel('Probability Density', fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend(fontsize=12)

# Add annotations for distribution parameters
plt.annotate(f'Posterior: Gamma(α={alpha_posterior}, β={beta_posterior})',
            xy=(0.05, 0.95), xycoords='axes fraction',
            bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="gray", alpha=0.8),
            fontsize=12)

plt.tight_layout()
plt.savefig('posterior_distribution_final.png', dpi=300)
plt.show()
```

```
Posterior distribution: Gamma(α'=49677, β'=10000)
MAP estimate: λ_MAP = 4.967600
Posterior mean: E[λ|data] = 4.967700
```

Code for Question 2, a):

```python
#Question 2, part a)

# Import the data
students_data = pd.read_csv('Students.csv')

# Print basic information
print("First 5 rows:")
print(students_data.head())
print("\nClass distribution:")
print(students_data['class'].value_counts())

# Create a figure with two subplots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10))

# Define colors and labels for consistency
colors = {'Class_1': 'red', 'Class_2': 'green', 'Unknown': 'gray'}
labels = {'Class_1': 'Failed', 'Class_2': 'Passed', 'Unknown': 'Unknown'}

# ------- Subplot 1: Histogram -------
for class_name, color in zip(['Class_1', 'Class_2', 'Unknown'], ['red', 'green', 'gray']):
    class_data = students_data[students_data['class'] == class_name]['x']
    if len(class_data) > 0:
        ax1.hist(class_data, bins=range(0, int(students_data['x'].max()) + 2),
                 alpha=0.5, color=colors[class_name],
                 label=f"{labels[class_name]} ({class_name})")

ax1.set_xlabel('Hours Studied per Week', fontsize=12)
ax1.set_ylabel('Number of Students', fontsize=12)
ax1.set_title('Distribution of Study Hours by Class', fontsize=14)
ax1.grid(True, alpha=0.3)
ax1.legend()

# ------- Subplot 2: Scatter Plot -------
# Add some jitter to the y-axis for better visualization
y_positions = {'Class_1': 1, 'Class_2': 2, 'Unknown': 3}

for class_name, group in students_data.groupby('class'):
    # Add small random jitter to y-position
    jitter = np.random.normal(0, 0.1, size=len(group))
    y_pos = [y_positions[class_name] + j for j in jitter]

    ax2.scatter(group['x'], y_pos,
                label=labels[class_name], color=colors[class_name], alpha=0.6)

ax2.set_xlabel('Hours Studied per Week', fontsize=12)
ax2.set_ylabel('Class', fontsize=12)
ax2.set_yticks([1, 2, 3])
ax2.set_yticklabels(['Failed (Class_1)', 'Passed (Class_2)', 'Unknown'])
ax2.set_title('Individual Students by Hours Studied and Class', fontsize=14)
ax2.grid(True, alpha=0.3)
ax2.legend()

plt.tight_layout()
plt.savefig('student_hours_combined.png', dpi=300)
plt.show()
```

Code for question 2, b):

```python
#Question 2, part b)

# Import the data
students_data = pd.read_csv('Students.csv')

# Filter data for Class_1 (failed students)
class1_data = students_data[students_data['class'] == 'Class_1']['x']

# Compute a and b (parameters of the uniform distribution)
a = class1_data.min()  # Minimum hours studied by failed students
b = class1_data.max()  # Maximum hours studied by failed students

print(f"Uniform distribution parameters for failed students:")
print(f"a (minimum) = {a}")
print(f"b (maximum) = {b}")

# Visualize the estimated uniform distribution against the actual data
plt.figure(figsize=(10, 6))

# Plot histogram of actual Class_1 data
plt.hist(class1_data, bins=range(int(a), int(b)+2),
         alpha=0.5, color='red', density=True,
         label='Actual data (Class_1)')

# Plot the estimated uniform distribution
x = np.linspace(a-0.5, b+0.5, 1000)
uniform_pdf = np.where((x >= a) & (x <= b), 1/(b-a), 0)
plt.plot(x, uniform_pdf, 'b-', linewidth=2,
         label=f'Uniform({a}, {b})')

plt.xlabel('Hours Studied per Week')
plt.ylabel('Probability Density')
plt.title('Estimated Uniform Distribution for Failed Students (Class_1)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('uniform_distribution_fit.png', dpi=300)
plt.show()
```

Code for question 2, c) :

```python
#Code for question 2, c)

# Import the data
students_data = pd.read_csv('Students.csv')

# Filter data for Class_2 (passed students)
class2_data = students_data[students_data['class'] == 'Class_2']['x']

# Compute μ (mean) and σ (standard deviation)
mu = class2_data.mean()
sigma = class2_data.std()

print(f"Normal distribution parameters for passed students:")
print(f"μ (mean) = {mu:.4f}")
print(f"σ (standard deviation) = {sigma:.4f}")

# Visualize the estimated normal distribution against the actual data
plt.figure(figsize=(10, 6))

# Plot histogram of actual Class_2 data
plt.hist(class2_data, bins=int(np.sqrt(len(class2_data))),
         alpha=0.5, color='green', density=True,
         label='Actual data (Class_2)')

# Plot the estimated normal distribution
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
normal_pdf = stats.norm.pdf(x, mu, sigma)
plt.plot(x, normal_pdf, 'b-', linewidth=2,
         label=f'Normal({mu:.2f}, {sigma:.2f}²)')

plt.xlabel('Hours Studied per Week')
plt.ylabel('Probability Density')
plt.title('Estimated Normal Distribution for Passed Students (Class_2)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('normal_distribution_fit.png', dpi=300)
plt.show()

# Optional: Perform a normality test to check if the data is normally distributed
k2, p_value = stats.normaltest(class2_data)
print(f"\nNormality test (D'Agostino's K²):")
print(f"K² statistic: {k2:.4f}")
print(f"p-value: {p_value:.4f}")
if p_value < 0.05:
    print("The data may not be normally distributed (p < 0.05)")
else:
    print("The data appears to be normally distributed (p >= 0.05)")
```

```
Normal distribution parameters for passed students:
μ (mean) = 8.9154
σ (standard deviation) = 1.9339
```

Code for question 2, d):

```python
#question 2, d)
# Import the data
students_data = pd.read_csv('Students.csv')

# Filter data for each class
class1_data = students_data[students_data['class'] == 'Class_1']['x']  # Failed students
class2_data = students_data[students_data['class'] == 'Class_2']['x']  # Passed students

# Compute parameters for uniform distribution (Class_1)
a = class1_data.min()
b = class1_data.max()

# Compute parameters for normal distribution (Class_2)
mu = class2_data.mean()
sigma = class2_data.std()

# Print the calculated parameters
print(f"Uniform distribution parameters (Class_1 - Failed):")
print(f"a = {a}, b = {b}")
print(f"\nNormal distribution parameters (Class_2 - Passed):")
print(f"μ = {mu:.4f}, σ = {sigma:.4f}")

# Create a range of x values for plotting
x = np.linspace(0, max(b, mu + 3*sigma) + 1, 1000)

# Calculate PDFs
uniform_pdf = np.where((x >= a) & (x <= b), 1/(b-a), 0)
normal_pdf = stats.norm.pdf(x, mu, sigma)

# Create the plot
plt.figure(figsize=(12, 7))

# Plot both density functions
plt.plot(x, uniform_pdf, 'r-', linewidth=2, label=f'P(x|z=1) ~ Uniform({a}, {b})')
plt.plot(x, normal_pdf, 'g-', linewidth=2, label=f'P(x|z=2) ~ Normal({mu:.2f}, {sigma:.2f}²)')

# Add histograms of actual data (optional, for comparison)
plt.hist(class1_data, bins=int(np.sqrt(len(class1_data))), density=True,
         alpha=0.3, color='red', label='Failed students (data)')
plt.hist(class2_data, bins=int(np.sqrt(len(class2_data))), density=True,
         alpha=0.3, color='green', label='Passed students (data)')

# Add labels and title
plt.xlabel('Hours Studied per Week', fontsize=12)
plt.ylabel('Probability Density', fontsize=12)
plt.title('Density Functions P(x|z=1) and P(x|z=2)', fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend()

# Add text annotations for the parameters
plt.text(0.02, 0.95, f"Failed: Uniform({a}, {b})", transform=plt.gca().transAxes,
         fontsize=12, verticalalignment='top', color='red')
plt.text(0.02, 0.90, f"Passed: Normal({mu:.2f}, {sigma:.2f}²)", transform=plt.gca().transAxes,
         fontsize=12, verticalalignment='top', color='green')

plt.tight_layout()
plt.savefig('class_densities.png', dpi=300)
plt.show()
```

```
Uniform distribution parameters (Class_1 - Failed):
a = 2.000655033542883, b = 9.99970797096162

Normal distribution parameters (Class_2 - Passed):
μ = 8.9154, σ = 1.9339
```



Code for question 2, e):

```
#code for question 2 part e continued...

plt.plot(x, uniform_pdf, 'r-', linewidth=2, label=f'P(x|z=1) ~ Failed')
plt.plot(x, normal_pdf, 'g-', linewidth=2, label=f'P(x|z=2) ~ Passed')

# Mark the unknown students on the plot
for res in results:
    marker = 'ro' if res['Prediction'] == 'Failed' else 'go'
    plt.plot(res['Hours'], 0.02, marker, markersize=10, alpha=0.7)
    plt.text(res['Hours'], 0.03, f"  Student {res['Student']}\n  {res['Hours']} hrs",
             verticalalignment='bottom')

# Add decision boundary
decision_boundary = None
for x_val in np.linspace(a, mu + 3*sigma, 1000):
    p1 = p_x_given_z1(x_val)
    p2 = p_x_given_z2(x_val)
    if abs(p1 - p2) < 0.0001:  # Approximately equal
        decision_boundary = x_val
        break

if decision_boundary:
    plt.axvline(x=decision_boundary, color='black', linestyle='--',
                label=f'Decision Boundary ({decision_boundary:.2f} hrs)')

plt.xlabel('Hours Studied per Week', fontsize=12)
plt.ylabel('Probability Density', fontsize=12)
plt.title('Classification of Unknown Students Based on Density Functions', fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend()

plt.tight_layout()
plt.savefig('unknown_classification.png', dpi=300)
plt.show()
```

Code for question 2, f) :

```python
#Question 2, f)

# First, load the data
students_data = pd.read_csv('Students.csv')

# Filter data for each class
class1_data = students_data[students_data['class'] == 'Class_1']['x']  # Failed
class2_data = students_data[students_data['class'] == 'Class_2']['x']  # Passed

# Calculate actual parameters (not example values)
# For uniform distribution (Failed students)
a = class1_data.min()
b = class1_data.max()
uniform_height = 1/(b-a)

# For normal distribution (Passed students)
mu = class2_data.mean()
sigma = class2_data.std()

print(f"Actual parameters from data:")
print(f"Uniform: a = {a}, b = {b}, height = {uniform_height}")
print(f"Normal: μ = {mu}, σ = {sigma}")

# Define the density functions with actual parameters
def uniform_pdf(x):
    return uniform_height if a <= x <= b else 0

def normal_pdf(x):
    return stats.norm.pdf(x, mu, sigma)

# Function to find where the difference between PDFs is zero
def pdf_difference(x):
    return uniform_pdf(x) - normal_pdf(x)

# Check specific value (7.7 hours)
hours_to_check = 7.7
p_uniform = uniform_pdf(hours_to_check)
p_normal = normal_pdf(hours_to_check)
print(f"\nAt x = {hours_to_check}:")
print(f"P({hours_to_check}|z=1) = {p_uniform:.6f}")
print(f"P({hours_to_check}|z=2) = {p_normal:.6f}")
print(f"Classification: {'Pass' if p_normal > p_uniform else 'Fail'}")

# Find the decision boundary using numerical method
try:
    # Search for crossing points in a wide range
    x_values = np.linspace(a-1, b+1, 1000)
    y_uniform = np.array([uniform_pdf(x) for x in x_values])
    y_normal = np.array([normal_pdf(x) for x in x_values])

    # Find indices where the sign changes
    sign_changes = np.where(np.diff(np.sign(y_uniform - y_normal)))[0]

    if len(sign_changes) > 0:
        # Use brentq to find exact crossing points
        decision_boundaries = []
        for idx in sign_changes:
            x_left = x_values[idx]
            x_right = x_values[idx + 1]
            x_c = brentq(pdf_difference, x_left, x_right)
            decision_boundaries.append(x_c)

        print(f"\nDecision boundary/boundaries:")
        for i, x_c in enumerate(decision_boundaries):
            print(f"x_c{i+1} = {x_c:.4f}")
            print(f"  P(x_c{i+1}|z=1) = {uniform_pdf(x_c):.6f}")
            print(f"  P(x_c{i+1}|z=2) = {normal_pdf(x_c):.6f}")
    else:
        print("\nNo crossing points found in the range examined.")
except Exception as e:
    print(f"Error in finding decision boundary: {e}")
```

```python
#Question 2, f) continued :

# Visualize with the correct parameters
plt.figure(figsize=(12, 7))
x = np.linspace(max(0, a-2), b+5, 1000)
y_uniform = np.array([uniform_pdf(x_val) for x_val in x])
y_normal = np.array([normal_pdf(x_val) for x_val in x])

plt.plot(x, y_uniform, 'r-', linewidth=2,
         label=f'P(x|z=1) ~ Failed: Uniform({a:.1f}, {b:.1f})')
plt.plot(x, y_normal, 'g-', linewidth=2,
         label=f'P(x|z=2) ~ Passed: Normal({mu:.1f}, {sigma:.1f}²)')

# Mark the decision boundaries if found
for i, x_c in enumerate(decision_boundaries):
    plt.axvline(x=x_c, color='black', linestyle='--',
                label=f'Decision Boundary {i+1}: x_c = {x_c:.4f}')

# Highlight the specific test point (7.7 hours)
plt.scatter([hours_to_check], [max(p_uniform, p_normal)], color='orange', s=100,
            zorder=5, label=f'Test point: {hours_to_check} hours')

plt.xlabel('Hours Studied per Week', fontsize=12)
plt.ylabel('Probability Density', fontsize=12)
plt.title('Decision Boundary with Actual Data Parameters', fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig('corrected_decision_boundary.png', dpi=300)
plt.show()
```

```
Actual parameters from data:
Uniform: a = 2.000655033542883, b = 9.99970797096162, height = 0.12501479960485123
Normal: μ = 8.915357953511139, σ = 1.9339379878295857

At x = 7.7:
P(7.7|z=1) = 0.125015
P(7.7|z=2) = 0.169320
Classification: Pass

Decision boundary/boundaries:
x_c1 = 2.0007
  P(x_c1|z=1) = 0.000000
  P(x_c1|z=2) = 0.000346
x_c2 = 6.9798
  P(x_c2|z=1) = 0.125015
  P(x_c2|z=2) = 0.125015
```

Code for Question 2, g):

```python
#Question 2, g)

# Calculate misclassification error
# Assuming we have decision boundaries from earlier calculation
x_c1 = 2.0007  # First decision boundary
x_c2 = 6.9798  # Second decision boundary
a = 2.0        # Lower bound of uniform distribution
b = 10.0       # Upper bound of uniform distribution

# Calculate misclassification error (false positive rate)
# For failed students, this is the probability they fall in the "classify as passed" regions
error_below = max(0, (x_c1 - a)/(b - a))  # Probability they fall below first boundary
error_above = max(0, (b - x_c2)/(b - a))  # Probability they fall above second boundary

misclassification_error = error_below + error_above
print(f"Misclassification error: {misclassification_error:.4f} or {misclassification_error*100:.2f}%")
```

```
Misclassification error: 0.3776 or 37.76%
```

Code for question 3, a):

```
[70]: #Question 3, a)

# Create a figure and axis
fig, ax = plt.subplots(figsize=(8, 6))

# Define node positions
positions = {
    'R': (0.5, 0.8),   # Ruth's disease status
    'J': (0.5, 0.5),   # James's disease status
    'S': (0.3, 0.2),   # James's sore throat
    'F': (0.7, 0.2)    # James's fever
}

# Define nodes
nodes = {
    'R': {'label': 'R\nRuth has disease\nP(R=1) = 1/4'},
    'J': {'label': 'J\nJames has disease\nP(J=1|R=1) = 3/4\nP(J=1|R=0) = 0'},
    'S': {'label': 'S\nSore throat\nP(S=1|J=1) = 1\nP(S=1|J=0) = 1/4'},
    'F': {'label': 'F\nElevated fever\nP(F=1|J=1) = 1/2\nP(F=1|J=0) = 1/4'}
}

# Draw nodes (circles with text)
for node, pos in positions.items():
    circle = plt.Circle(pos, 0.1, fill=True, color='skyblue', alpha=0.7)
    ax.add_patch(circle)
    ax.text(pos[0], pos[1], node, ha='center', va='center', fontsize=14, fontweight='bold')
    ax.text(pos[0], pos[1]-0.15, nodes[node]['label'], ha='center', va='center', fontsize=9)

# Draw directed edges
edges = [('R', 'J'), ('J', 'S'), ('J', 'F')]
for start, end in edges:
    start_pos = positions[start]
    end_pos = positions[end]

    # Calculate direction vector
    dx = end_pos[0] - start_pos[0]
    dy = end_pos[1] - start_pos[1]
    length = np.sqrt(dx**2 + dy**2)

    # Adjust start and end points to be at the edge of the nodes
    start_x = start_pos[0] + 0.1 * dx/length
    start_y = start_pos[1] + 0.1 * dy/length
    end_x = end_pos[0] - 0.1 * dx/length
    end_y = end_pos[1] - 0.1 * dy/length

    # Draw arrow
    ax.arrow(start_x, start_y, end_x-start_x, end_y-start_y, head_width=0.03,
             head_length=0.05, fc='black', ec='black', length_includes_head=True)

# Set axis limits and remove ticks
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_xticks([])
ax.set_yticks([])

# Add title and box around the graph
ax.set_title('Directed Acyclic Graph: Disease Transmission Model', fontsize=14)
ax.spines['top'].set_visible(True)
ax.spines['right'].set_visible(True)
ax.spines['bottom'].set_visible(True)
ax.spines['left'].set_visible(True)

plt.tight_layout()
plt.savefig('disease_dag.png', dpi=300)
plt.show()
```

Code for question 3, b):

```
[101]: #Question 3, b)

# Display the joint probability factorization
display(Math(r'\text{Joint Probability Factorization:}'))
display(Math(r'P(R, J, S, F) = P(R) \times P(J|R) \times P(S|J) \times P(F|J)'))

# Display all probabilities given in the question
display(Math(r'\text{Probabilities Given in the Question:}'))

# Prior probability for Ruth
display(Math(r'\text{Prior Probability:}'))
display(Math(r'P(R = 1) = \frac{1}{4}'))
display(Math(r'P(R = 0) = \frac{3}{4}'))

# Conditional probability for James given Ruth's status
display(Math(r'\text{Probability of James Being Infected:}'))
display(Math(r'P(J = 1|R = 1) = \frac{3}{4}'))
display(Math(r'P(J = 0|R = 1) = \frac{1}{4}'))
display(Math(r'P(J = 1|R = 0) = 0'))
display(Math(r'P(J = 0|R = 0) = 1'))

# Conditional probability for Sore Throat given James's status
display(Math(r'\text{Probability of Sore Throat:}'))
display(Math(r'P(S = 1|J = 1) = 1'))
display(Math(r'P(S = 0|J = 1) = 0'))
display(Math(r'P(S = 1|J = 0) = \frac{1}{4}'))
display(Math(r'P(S = 0|J = 0) = \frac{3}{4}'))

# Conditional probability for Fever given James's status
display(Math(r'\text{Probability of Elevated Fever:}'))
display(Math(r'P(F = 1|J = 1) = \frac{1}{2}'))
display(Math(r'P(F = 0|J = 1) = \frac{1}{2}'))
display(Math(r'P(F = 1|J = 0) = \frac{1}{4}'))
display(Math(r'P(F = 0|J = 0) = \frac{3}{4}'))

# Example joint probability calculation
display(Math(r'\text{Example Calculation:}'))
display(Math(r'P(R=1, J=1, S=1, F=1) = P(R=1) \times P(J=1|R=1) \times P(S=1|J=1) \times P(F=1|J=1)'))
display(Math(r'P(R=1, J=1, S=1, F=1) = \frac{1}{4} \times \frac{3}{4} \times 1 \times \frac{1}{2} = \frac{3}{32}'))
```

Code for question 3, c):

```
•[118]: #Question 3, c)

# calculate P(S=1, F=1|R=1)
display(Math(r'\text{Step 1 : Calculate } P(S=1, F=1|R=1)'))
display(Math(r'P(S=1, F=1|R=1) = P(S=1|J=1) \times P(F=1|J=1) \times P(J=1|R=1) + P(S=1|J=0) \times P(F=1|J=0) \times P(J=0|R=1)'))
display(Math(r'P(S=1, F=1|R=1) = 1 \times \frac{1}{2} \times \frac{3}{4} + \frac{1}{4} \times \frac{1}{4} \times \frac{1}{4}'))
display(Math(r'P(S=1, F=1|R=1) = \frac{3}{8} + \frac{1}{64} = \frac{24}{64} + \frac{1}{64} = \frac{25}{64}'))

# calculate P(S=1, F=1|R=0) - this was correct
display(Math(r'\text{Step 2: Calculate } P(S=1, F=1|R=0)'))
display(Math(r'P(S=1, F=1|R=0) = 1 \times \frac{1}{2} \times 0 + \frac{1}{4} \times \frac{1}{4} \times 1 = \frac{1}{16}'))

# calculate P(S=1, F=1)
display(Math(r'\text{Step 3 : Calculate } P(S=1, F=1)'))
display(Math(r'P(S=1, F=1) = P(S=1, F=1|R=1) \times P(R=1) + P(S=1, F=1|R=0) \times P(R=0)'))
display(Math(r'P(S=1, F=1) = \frac{25}{64} \times \frac{1}{4} + \frac{1}{16} \times \frac{3}{4}'))
display(Math(r'P(S=1, F=1) = \frac{25}{256} + \frac{3}{64} = \frac{25}{256} + \frac{12}{256} = \frac{37}{256}'))

# Apply Bayes' theorem with  values
display(Math(r'\text{Step 4 : Apply Bayes\' theorem}'))
display(Math(r'P(R=1|S=1, F=1) = \frac{P(S=1, F=1|R=1) \times P(R=1)}{P(S=1, F=1)}'))
display(Math(r'P(R=1|S=1, F=1) = \frac{\frac{25}{64} \times \frac{1}{4}}{\frac{37}{256}}'))
display(Math(r'P(R=1|S=1, F=1) = \frac{\frac{25}{256}}{\frac{37}{256}} = \frac{25}{37}'))

# Highlight the final answer
display(Math(r'\boxed{P(R=1|S=1, F=1) = \frac{25}{37}}'))
```

Code for question 3, d):

```
#Question 3, d)

# Display the question
display(Math(r'\text{Find } P(R=1|S=0, F=1)'))

# Start with Bayes' theorem
display(Math(r'P(R=1|S=0, F=1) = \frac{P(S=0, F=1|R=1) \times P(R=1)}{P(S=0, F=1)}'))

# Calculate P(S=0, F=1|R=1)
display(Math(r'\text{Step 1: Calculate } P(S=0, F=1|R=1)'))
display(Math(r'P(S=0, F=1|R=1) = \sum_{j \in \{0,1\}} P(S=0|J=j) \times P(F=1|J=j) \times P(J=j|R=1)'))
display(Math(r'P(S=0, F=1|R=1) = P(S=0|J=1) \times P(F=1|J=1) \times P(J=1|R=1) + P(S=0|J=0) \times P(F=1|J=0) \times P(J=0|R=1)'))
display(Math(r'P(S=0, F=1|R=1) = 0 \times \frac{1}{2} \times \frac{3}{4} + \frac{3}{4} \times \frac{1}{4} \times \frac{1}{4}'))
display(Math(r'P(S=0, F=1|R=1) = 0 + \frac{3 \times 1 \times 1}{4 \times 4 \times 4} = \frac{3}{64}'))

# Calculate P(S=0, F=1|R=0)
display(Math(r'\text{Step 2: Calculate } P(S=0, F=1|R=0)'))
display(Math(r'P(S=0, F=1|R=0) = P(S=0|J=1) \times P(F=1|J=1) \times P(J=1|R=0) + P(S=0|J=0) \times P(F=1|J=0) \times P(J=0|R=0)'))
display(Math(r'P(S=0, F=1|R=0) = 0 \times \frac{1}{2} \times 0 + \frac{3}{4} \times \frac{1}{4} \times 1'))
display(Math(r'P(S=0, F=1|R=0) = 0 + \frac{3}{16} = \frac{3}{16}'))

# Calculate P(S=0, F=1)
display(Math(r'\text{Step 3: Calculate } P(S=0, F=1)'))
display(Math(r'P(S=0, F=1) = P(S=0, F=1|R=1) \times P(R=1) + P(S=0, F=1|R=0) \times P(R=0)'))
display(Math(r'P(S=0, F=1) = \frac{3}{64} \times \frac{1}{4} + \frac{3}{16} \times \frac{3}{4}'))
display(Math(r'P(S=0, F=1) = \frac{3}{256} + \frac{9}{64} = \frac{3}{256} + \frac{36}{256} = \frac{39}{256}'))

# Apply Bayes' theorem
display(Math(r'\text{Step 4: Apply Bayes\' theorem}'))
display(Math(r'P(R=1|S=0, F=1) = \frac{P(S=0, F=1|R=1) \times P(R=1)}{P(S=0, F=1)}'))
display(Math(r'P(R=1|S=0, F=1) = \frac{\frac{3}{64} \times \frac{1}{4}}{\frac{39}{256}}'))
display(Math(r'P(R=1|S=0, F=1) = \frac{\frac{3}{256}}{\frac{39}{256}} = \frac{3}{39} = \frac{1}{13}'))

# Highlight the final answer
display(Math(r'\boxed{P(R=1|S=0, F=1) = \frac{1}{13}}'))
```

Code for question 3, e):

```
[122]: #Question 3, e)

# Display the question
display(Math(r'\text{Find } P(J=1|F=1, T=1) \text{ (probability James is ill given fever and Ruth\'s positive test)}'))

# Apply Bayes' theorem
display(Math(r'P(J=1|F=1, T=1) = \frac{P(F=1|J=1, T=1) \times P(J=1|T=1)}{P(F=1|T=1)}'))

# Step 1: Find P(R=1|T=1) using Bayes' theorem
display(Math(r'\text{Step 1: Calculate } P(R=1|T=1) \text{ (probability Ruth has disease given positive test)}'))
display(Math(r'P(R=1|T=1) = \frac{P(T=1|R=1) \times P(R=1)}{P(T=1)}'))
display(Math(r'P(T=1) = P(T=1|R=1) \times P(R=1) + P(T=1|R=0) \times P(R=0)'))
display(Math(r'P(T=1) = \frac{3}{4} \times \frac{1}{4} + \frac{1}{4} \times \frac{3}{4} = \frac{3}{16} + \frac{3}{16} = \frac{6}{16} = \frac{3}{8}'))
display(Math(r'P(R=1|T=1) = \frac{\frac{3}{4} \times \frac{1}{4}}{\frac{3}{8}} = \frac{\frac{3}{16}}{\frac{3}{8}} = \frac{1}{2}'))

# Step 2: Find P(J=1|T=1)
display(Math(r'\text{Step 2: Calculate } P(J=1|T=1) \text{ (probability James is ill given Ruth\'s positive test)}'))
display(Math(r'P(J=1|T=1) = P(J=1|R=1) \times P(R=1|T=1) + P(J=1|R=0) \times P(R=0|T=1)'))
display(Math(r'P(J=1|T=1) = \frac{3}{4} \times \frac{1}{2} + 0 \times \frac{1}{2} = \frac{3}{8}'))

# Step 3: Find P(F=1|T=1) using law of total probability
display(Math(r'\text{Step 3: Calculate } P(F=1|T=1) \text{ (probability James has fever given Ruth\'s positive test)}'))
display(Math(r'P(F=1|T=1) = P(F=1|J=1) \times P(J=1|T=1) + P(F=1|J=0) \times P(J=0|T=1)'))
display(Math(r'P(F=1|T=1) = \frac{1}{2} \times \frac{3}{8} + \frac{1}{4} \times \frac{5}{8} = \frac{3}{16} + \frac{5}{32} = \frac{6}{32} + \frac{5}{32} =

# Step 4: Apply Bayes' theorem to find P(J=1|F=1, T=1)
display(Math(r'\text{Step 4: Apply Bayes\' theorem}'))
display(Math(r'P(J=1|F=1, T=1) = \frac{P(F=1|J=1) \times P(J=1|T=1)}{P(F=1|T=1)}'))
display(Math(r'P(J=1|F=1, T=1) = \frac{\frac{1}{2} \times \frac{3}{8}}{\frac{11}{32}} = \frac{\frac{3}{16}}{\frac{11}{32}} = \frac{3}{16} \times \frac{32

# Highlight the final answer
display(Math(r'\boxed{P(J=1|F=1, T=1) = \frac{6}{11}}'))
```

This entire project was completed using python on Jupyter notebook.

Packages used for this Project :

```python
# Packages needed for this project
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from scipy.optimize import brentq
import matplotlib.patches as patches
import math
from IPython.display import display, Math
```